

【Hello Agent 学习笔记】第七章

从0 构建智能体框架

📁 Channel

本章讲了什么

- 文章从零构建了一个基础的智能体框架。这框架支持多种基础功能，比如对多种模型供应商的支持，对于配置的统一管理，以及对于工具的结构化管理和引入。
- 基于框架实现前面章节所涉及到的几种 agent，包括 ReAct，Plan&Solve，Reflection
- 对工具类进行封装，通过标准化的工具注册机制来管理新工具的注册和调用，支持工具类和工具函数。

知识点讨论



在实际应用中，不同的服务商在环境变量命名、默认 API 地址和推荐模型等方面都存在差异。如果每次切换服务商都需要用户手动查询并修改代码，会极大影响开发效率。

框架的意义就是解决同一类应用开发过程中遇到的相似问题。对于现在百家争鸣的大模型来说，选用和调试不同的大模型的底层能力提供者是框架层面解决的基础需求。

这一部分的设计主要是通过环境变量的识别和URL的预先定义来实现对于各个不同模型的统一支持，让模型的调用变得比较简单。现在的前提还是在于它内部封装了对于各个不同模型供应商的API协议的支持，并且通过引入额外的自定义的 LLM 调用函数，给Agent开发的内部做出了一致的接口。



Framework Interface：Agent 需要一系列配套的接口和组件来处理数据流、管理配置、应对异常，并为上层应用的构建提供一个清晰、统一的结构。

Agent 作为新的应用范式，它的本质是处理用户跟 LLM 之间的信息交互，那信息作为核心的处理数据，需要通过统一、标准化的结构来进行管理，方便进行行为的追踪和问题的排查。

而配置系统将通用的默认配置封装起来，让agent 可以方便的基于预设的最佳实践来快速的开始开发。



```
# 7.4.1
class MySimpleAgent(SimpleAgent):
    # ...
    def run(self, input_text: str, max_tool_iterations: int = 3, **kwargs) -> str:
        # ...
        # 添加系统消息 (可能包含工具信息)
        enhanced_system_prompt = self._get_enhanced_system_prompt()
        messages.append({"role": "system", "content": enhanced_system_prompt})
        # ...

    def _get_enhanced_system_prompt(self) -> str:
        """构建增强的系统提示词，包含工具信息"""
        base_prompt = self.system_prompt or "你是一个有用的AI助手。"
        # ...
        # 获取工具描述
        tools_description = self.tool_registry.get_tools_description()
        tools_section = "\n\n## 可用工具\n"
        tools_section += "你可以使用以下工具来帮助回答问题:\n"
        tools_section += tools_description + "\n"

        tools_section += "\n## 工具调用格式\n"
        tools_section += "当需要使用工具时，请使用以下格式:\n"
        tools_section += "`[TOOL_CALL:{tool_name}:{parameters}]`\n"
        tools_section += "例如: `[TOOL_CALL:search:Python编程]` 或 `[TOOL_CALL:memory:recall=用户信息]`\n\n"
        tools_section += "工具调用结果会自动插入到对话"
```

```
中，然后你可以基于结果继续回答。\\n"  
    return base_prompt + tools_section
```

看这段代码你可以看到所有的 tool 的信息是被整个的加载到system prompt当中的。这样做出现的问题就是当agent 的 tool 的引用越来越多的时候，system prompt会被撑得特别大，那每一次和llm进行交互的时候，这些系统提示词都会被默认的传递过去，导致token的极大的浪费。

改进的方法有很多，最常见的就是像最近比较火的skill，它将所有的工具分层进行定义。在调用大模型的时候，只传输给最上层最抽象的工具的调用基础信息，告诉大模型在什么情况下使用这个skill，使用这个工具。那具体使用的时候怎么样操作，在后续真正需要使用的时候再传给LLM，这样子就避免了每次都传输给大模型工具的详细调用方式。



4.5.4.1

工具链式调用机制

```
def create_research_chain() -> ToolChain:
    """创建一个研究工具链:搜索 -> 计算 -> 总结"""
    chain = ToolChain(
        name="research_and_calculate",
        description="搜索信息并进行相关计算"
    )
    # 步骤1:搜索信息
    chain.add_step(
        tool_name="search",
        input_template="{input}",
        output_key="search_result"
    )
    # 步骤2:基于搜索结果进行计算 (如果需要)
    chain.add_step(
        tool_name="my_calculator",
        input_template="根据以下信息计算相关数值:{search_result}",
        output_key="calculation_result"
    )
    return chain
```

注意看这个示例，其实可以看到在步骤二的input template里边会用到search_result这个关键字，这个关键字其实是 步骤1 的output key，这就表明了当我们使用ToolChain的时候，每一步的先后输入关系是需要再通过提示词以及变量的精确匹配来进行关联的，否则这样的步骤前后关系是无法关联起来的。

在实际进行工业化的开发的过程中，这样的先后依赖关系是非常脆弱的。我们需要更精确的先后结构化变量的匹配来进行关联，或者关联关系通过用户透明的方式、预先配置的方式、动态注入的方式，关联到每一步的input当中，避免不精确的匹配导致的错误。

碎碎念

通过基础的框架设计与实现，我们学习到了在框架使用过程中经常需要解决的问题，但其中的代码质量以及所交付的代码只是一些实验性的脚手架，它对于真正工业化的落地之间还有很大的距离，所以目前这些代码只存在于能跑的阶段。那如果我们想要使用和自己设计更完善，甚至说是更健壮更稳定的智能体，可以去看一下工业上面的比较流行的agent框架来去进行学习。

但其实学习过程当中我也了解到了，跑通本身就是一种学习的方式，而不用完全完善的设计。就像学习一个新东西的过程一样，如果你对这个东西不是很了解，对于agent的行为方式、行为模式和具体的应用场景不是特别熟悉的话，那就做边学，先把场景跑通再去优化，逐步对遇到的问题进行分析和改善，才是一个比较高效率的学习方式。

有兴趣加入的同学可以留言“一起”，拉你加入学习小组共同学习讨论~